

Etapa 1 IDP - Musichology

1. Membrii:

Constantinescu Vlad 343C3

Popescu Matei 342C4

2. Asistent responsabil:

Ciobanu Radu

3. Descrierea tematicii și funcționalităților

Musichology este o platformă cloud native concepută ca un asistent de terapie muzicală digitală, ce utilizează microservicii pentru a corela profilul emoțional al utilizatorilor cu recomandări muzicale personalizate.

Sistemul implementează o abordare avansată de analiză multimodală, transformând input-ul brut în coordonate matematice precise:

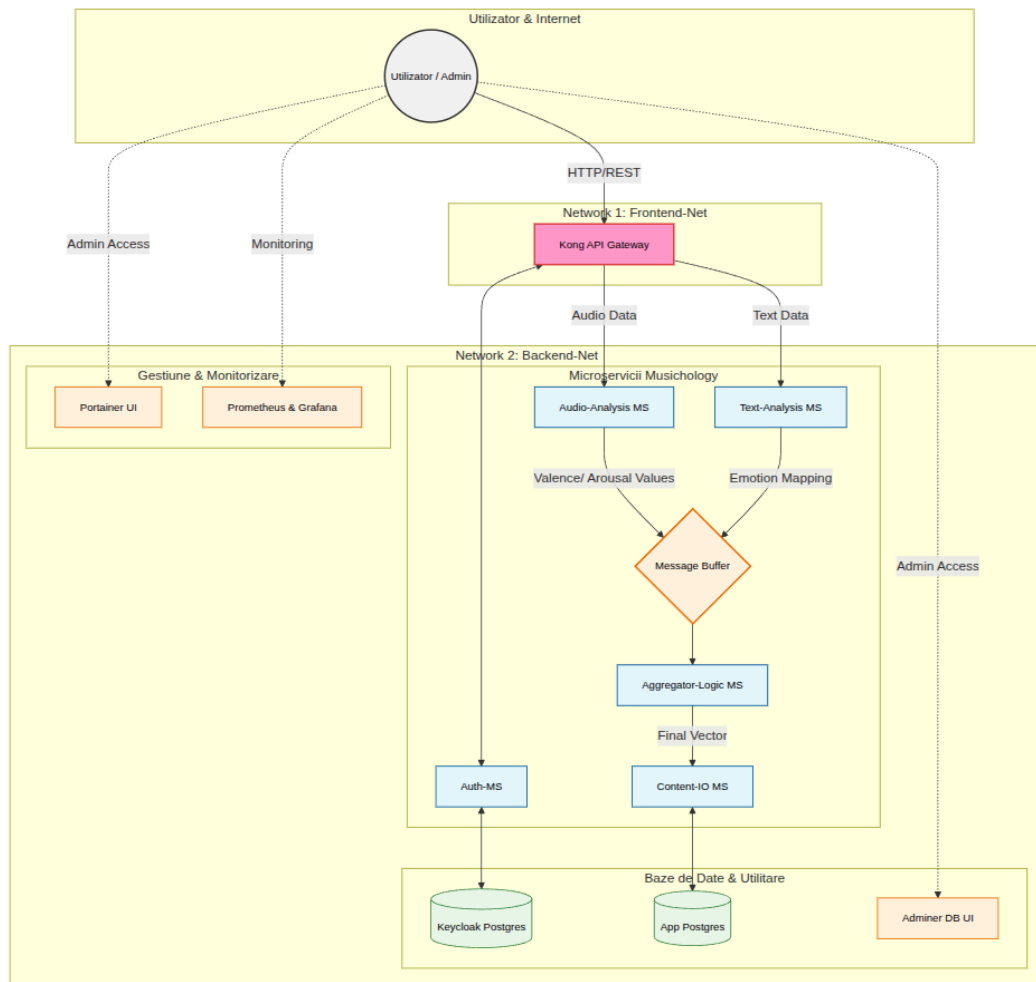
- **Analiză Emoțională Duală (Pipeline Asincron):**

- **Analiză Audio:** Sistemul procesează fișiere audio prin extracție de caracteristici de frecvență, utilizând o rețea neuronală dedicată pentru a genera un embedding ce poziționează starea pe axele **Valence** (pozitivitate) și **Arousal** (energie).
- **Analiză Textuală:** Versurile sau descrierile sunt procesate prin pipeline-uri de **NLP**. Sistemul clasifică textul în emoțiile de bază (Ekman) și mapează emoția dominantă în același spațiu dimensional (V-A).

- **Sistem de Fuziune și Recomandare:** Rezultatele din ambele fluxuri sunt sincronizate într-un **buffer de mesaje**, de unde un serviciu de agregare calculează un vector emoțional final (fused vector).

- **Monitorizare și Evoluție Emoțională:** Platforma stochează istoricul interacțiunilor, permițând vizualizarea progresului și a schimbărilor de stare prin dashboard-uri de observabilitate.

4. Arhitectura aplicației



5. Descrierea componentelor și a tehnologiilor folosite

1. Microservicii Proprii (Core Business Logic)

Acestea sunt unitățile funcționale autonome care procesează datele brute și le convertesc într-un model emoțional dimensional (Valence-Arousal):

- **Auth-MS:** Serviciu dedicat securității, care integrează **Keycloak**. Gestionează identitățile, permisiunile bazate pe roluri (RBAC) și emite token-uri **JWT** necesare pentru autorizarea cererilor în restul microserviciilor.
- **Audio-Analysis-MS (Neural Network):** Microserviciu care procesează fișierele audio prin extracția de caracteristici. Acestea sunt introduse într-o **rețea neuronală** pentru a genera coordonate matematice pe axele **Valence** (pozitivitate) și **Arousal** (energie).
- **Text-Analysis-MS (NLP):** Componentă care analizează versurile sau descrierile textuale. Utilizează un pipeline de **NLP** pentru clasificarea input-ului în cele 6 emoții de bază, mapând ulterior emoția dominantă pe modelul dimensional de emoții pentru a fi compatibilă cu analiza audio.
- **Aggregator-Logic-MS (Vector Fusion):** Reprezintă nucleul decizional. Preia rezultatele parțiale din cele două module anterioare și realizează o fuziune vectorială (fused analysis) pentru a stabili punctul exact al stării utilizatorului în spațiul bidimensional.
- **Content-IO-MS (Data Interaction):** Asigură izolarea bazei de date de logica de business. Este singura componentă care interacționează cu DB-ul principal pentru a salva istoricul și a interoga biblioteca muzicală.

2. Componente Suport (Infrastructure & Storage)

- **Baza de Date de Autentificare (PostgreSQL):** Instanță izolată utilizată exclusiv pentru persistența datelor de securitate ale Keycloak.
- **Baza de Date Principală (PostgreSQL):** Stochează metadatele melodiilor și vectorii lor emoționali, precum și informațiile despre utilizatori, profilor și istoricul muzical.
- **Kong API Gateway:** Punctul unic de intrare în cluster care se ocupă de rutarea traficului, load balancing și securitatea la nivel de edge.
- **Adminer & Portainer:** Utilitare pentru gestiunea vizuală: Adminer pentru administrarea tabelor SQL și Portainer pentru managementul containerelor și al serviciilor din cluster.
- **Stack de Monitorizare (Prometheus & Grafana):** Asigură observabilitatea sistemului prin dashboard-uri ce urmăresc starea serviciilor și latența procesării.

3. Orchestrare și Deployment

- **Docker Swarm:** Infrastructura de tip cluster pe care este orchestrat întreg stack-ul de servicii, asigurând scalabilitatea orizontală a componentelor de analiză.
- **GitLab CI/CD:** Instrumentul care automatizează pipeline-ul de build, test și deployment, asigurând livrarea continuă a codului în mediul de producție.

6. Responsabilităților fiecărui membru al echipei în cadrul proiectului

Popescu Matei

- **Procesare Semnal și AI:** Dezvoltarea microserviciului **Audio-Analysis-MS**, implementarea rețelei neuronale pentru extracția caracteristicilor acustice și maparea acestora pe axele *Valence-Arousal*.
- **Logică de Fuziune:** Implementarea microserviciului **Aggregator-Logic-MS**, responsabil pentru preluarea datelor din buffer și calcularea vectorului emoțional final prin fuziunea rezultatelor audio-text.
- **Persistență Date:** Configurarea și administrarea **Bazei de Date Principale (PostgreSQL)**, incluzând definirea schemei pentru biblioteca muzicală și optimizarea căutărilor vectoriale.
- **Observabilitate și Gestiune DB:** Implementarea stack-ului de monitorizare (**Prometheus & Grafana**) pentru urmărirea performanței sistemului și configurarea **Adminer** pentru managementul vizual al datelor.

Constantinescu Vlad

- **Securitate și Identitate:** Implementarea **Auth-MS** și integrarea cu **Keycloak**, gestionând baza de date de autentificare, fluxurile de login și autorizarea bazată pe roluri (RBAC) prin token-uri JWT.
- **Analiză Textuală:** Dezvoltarea microserviciului **Text-Analysis-MS**, utilizând pipeline-uri de **NLP** pentru clasificarea emoțiilor din versuri/text și transpunerea acestora în modelul dimensional.
- **Gestiune Flux Date:** Implementarea microserviciului **Content-IO-MS**, asigurând interfața securizată între logica de business și baza de date principală (operațiuni CRUD).
- **Persistență Date:** Configurarea și administrarea **Bazei de Date Principale (PostgreSQL)**, incluzând definirea schemei pentru biblioteca muzicală și optimizarea căutărilor vectoriale.
- **Infrastructură și Gateway:** Configurarea **Kong API Gateway** pentru rutarea externă a serviciilor și utilizarea **Portainer** pentru orchestrarea și monitorizarea containerelor în clusterul Docker Swarm.

7. [Repository](#)